Multiplayer Game Platform — Initial Project Plan

Developed by Bogo Solutions

Overview

Three one-week sprints covering the full implementation of the platform as designed in the architecture document. Each sprint is organized by sub-team: **Platform Core**, **Client/UI**, **Rules & Validation**, and **Quality, Testing & Integration**.

Sprint 1 builds the foundation. Sprint 2 implements the core platform features. Sprint 3 integrates everything, closes out stats and notifications, and prepares the demo.

Branch naming convention: Each backlog ID corresponds to a single feature branch using the format `ID/short-description`, e.g., `GAME-01/game-interfaces`. All branches merge into `develop`; `develop` merges into `main` at sprint boundaries via a reviewed merge request. No backlog ID spans more than one sprint.

Table of Contents

1. Sprint 1 — Foundation & Core Abstractions
2. Sprint 2 — Core Platform Features
3. Sprint 3 — Integration, Notifications & Demo
4. Dependency Summary
5. Product Backlog

Sprint 1 — Foundation & Core Abstractions (Week 1)

Goal: Establish the project skeleton, all shared interfaces and data models, stubs, and the TCP network boundary so every sub-team can work in parallel from Week 2 onward.

Platform Core

Task	Backlog ID
Project structure setup (`ui`, `platform`, `game`, `model` packages)	INFRA-01
`NETWORK` stub — TCP socket boundary (`send` / `receive`, same IP/different ports)	STUBS-01
`Database` stub (`storeData`, `retrieveData`)	STUBS-01

Task	Backlog ID
`CredentialsDatabase` stub (login boolean, register)	STUBS-01
`ClientCore` skeleton — method stubs only (`send` , `receive` , `check`)	CLIENT-01
`ClientReceiver` skeleton — listen loop stub, message type dispatch stub	CLIENT-01
`ServerCore` skeleton — method stubs only (`send` , `receive` , `routeContent`)	SERVER-01
`GameStateUpdate` event class (`sessionId` , `recipients` , `status` , `activePlayerId` , `payload`)	EVENTS-01
`GameStatus` enum (`LOBBY` , `MATCHMAKING` , `STARTING` , `IN_PROGRESS` , `GAME_OVER`)	EVENTS-01

Client/UI

Task	Backlog ID
`WelcomePage`	UI-01
`AuthenticationScreen`	UI-01
`LoginPage`	UI-01
`State` enum (login/register error states: `UsernameTaken` , `EmailFormatWrong` , `VerificationCodeWrong`)	UI-01

Rules & Validation

Task	Backlog ID
`Game` interface (`startGame` , `handleAction` , `validateAction` , `getGameState` , `getGameUpdate` , `isGameOver`)	GAME-01
`RuleValidator` interface (`validate(PlayerAction, GameState): ValidationResult` , `getValidActions(GameState)`)	GAME-01
`PlayerAction` abstraction (`playerId` , `actionType`)	GAME-01
`clientGame` interface (`validate(move): Bool`) — client-side validation hook consumed by `ClientCore`	GAME-01
`Player` domain model	MODEL-

Task	Backlog ID
	01
`GeneralStats` class	MODEL-01
`Rank` enum	MODEL-01
`PlayerStats` class	MODEL-01
`Settings` class	MODEL-01

Quality, Testing & Integration

Task	Backlog ID
GitLab repository setup, README skeleton, branching conventions documented	INFRA-01
Test infrastructure setup (JUnit, test runner config)	QA-01
Unit tests: `Player`, `GeneralStats`	QA-01
Unit tests: `CredentialsDatabase` stub	QA-01
Unit tests: `NETWORK` stub send/receive	QA-01
Unit tests: `GameStateUpdate`, `GameStatus`	QA-01

Sprint 2 — Core Platform Features (Week 2)

Goal: Implement the full request-routing pipeline (`ClientCore → NETWORK → ServerCore`), wire up `GameLobby`, `matchmaking`, `Chat`, and the `Game` service skeleton. Implement both game class structures and the primary UI screens. By end of this sprint the lobby → game start → turn progression flow should be runnable end-to-end.

Platform Core

Task	Backlog ID
`ClientCore` — full implementation (local request handling, TCP dispatch, `clientGame.check` integration)	CLIENT-02

Task	Backlog ID
`ClientReceiver` — full implementation (server-push listener thread, update type detection, UI routing)	CLIENT-02
`ServerCore` — full routing (`receiveMessage` , `routeContent` to `GameLobby` , `Chat` , `Database`)	SERVER-02
`SessionManager` — full implementation (create, store, invalidate session tokens on login/logout)	AUTH-01
`GameLobby` service (`receiveContent` , `requestJoin` , display game info, hand off to `matchmaking`)	LOBBY-01
`matchmaking` service (queue management, match-found routing, new `Game` instance creation)	MATCH-01
`Chat` service (`receiveContent` → `sendMessage` , `List<int>` user list management)	CHAT-01
`Game` service skeleton (turn update routing stubs, move dispatch to `GameEngine` stubs)	SESSION-01
`GameEngine` interface (`validateMove(move): Boolean` , `updateGame(move)`)	SESSION-01
`Leaderboard` service skeleton (`logGameResult` , `updatePlayer` stubs, `Database` stub wired)	LEADER-01

Client/UI

Task	Backlog ID
`HomePage`	UI-02
`MainGameDashboard` (game selection, queue entry, leaderboard placeholder)	UI-02
`ProfileDashboard`	UI-03
`PlayGame` (calls `ClientCore.send`)	UI-04
`GameGUI` (`renderBoard` , `updateUI`)	UI-04
`ChatboxGUI`	UI-04

Rules & Validation

Task	Backlog ID
`TicTacToeGame` (implements `Game`)	GAME-02

Task	Backlog ID
`TicTacToePlayerAction` (implements `PlayerAction`, carries `position: Position`)	GAME-02
`TicTacToeRuleValidator` (implements `RuleValidator`, returns `ValidationResult`)	GAME-02
`TicTacToeGameState` (`board: Symbol[3][3]`, `activePlayerId`, `moveCount`, `status`)	GAME-02
`ConnectFourGame` (implements `Game`)	GAME-03
`ConnectFourPlayerAction` (implements `PlayerAction`, carries `move: int` column index)	GAME-03
`ConnectFourRuleValidator` (implements `RuleValidator`, validates column not full)	GAME-03
`ConnectFourGameState` (`board: Player[6][7]`, `playerTurn`, `state`)	GAME-03

Quality, Testing & Integration

Task	Backlog ID
Merge all Sprint 1 branches into `develop`, end-to-end smoke test	QA-02
Architecture doc updated to reflect Sprint 2 changes	QA-03
Unit tests: `ClientCore`, `ClientReceiver`, `ServerCore` routing	QA-04
Unit tests: `GameLobby`, `matchmaking`	QA-04
Unit tests: `TicTacToeRuleValidator`, `ConnectFourRuleValidator`	QA-04
Integration test: login flow (`ClientCore` → `NETWORK` → `ServerCore` → `CredentialsDatabase` stub)	QA-04
Integration test: lobby join flow (`ClientCore` → `ServerCore` → `GameLobby` → `matchmaking`)	QA-04

Sprint 3 — Integration, Notifications & Demo (Week 3)

Goal: Complete the move pipeline and notification system end-to-end (`GameSessionManager` → `ServerCore` → `ClientReceiver` → UI). Close out game-over handling, stats, leaderboard, and all remaining UI. Verify and deliver the demo.

Platform Core

Task	Backlog ID
`GameSessionManager` — full move pipeline (`validateMove` → `updateGame` → check end conditions → publish `GameStateUpdate`)	SESSION-02
`Game` service — full implementation (move pipeline, state broadcast to both players via `ServerCore` , local chat, return-to-lobby)	SESSION-02
`Event Management` — full wiring (`GameSessionManager` publishes, `ServerCore` subscribes and routes to `ClientReceiver`)	SESSION-02
`StatsManager` (aggregates `GeneralStats` + `Rank` , persists via `Database` stub after game-over)	STATS-01
`Leaderboard` service — full implementation (`getTopPlayersByRank` , MMR recalculation in `updatePlayer`)	STATS-01

Client/UI

Task	Backlog ID
Turn alert display (receives `GameStateUpdate(yourTurn)` via `ClientReceiver`)	UI-05
Win/loss/tie result screen (receives `GameStateUpdate(gameOver)` , displays outcome)	UI-05
`Settings` UI (`changeUsername` , `changeEmail` , `changePassword` , `logout`)	UI-06
Player statistics view (wired to `ProfileDashboard`)	UI-06
Leaderboard view (wired to `MainGameDashboard`)	UI-06
Full navigation wiring & UI polish (all screen transitions verified end-to-end)	UI-06

Rules & Validation

Task	Backlog ID
Win detection — Tic-Tac-Toe (3-in-a-row, all directions)	GAME-04
`clientGame` implementation for Tic-Tac-Toe (wired into `ClientCore.check`)	GAME-04
`ValidationResult.failureReason` propagation to UI via `GameStateUpdate` — Tic-Tac-Toe	GAME-04
Win detection — Connect Four (4-in-a-row, horizontal, vertical, diagonal)	GAME-05
`clientGame` implementation for Connect Four (wired into `ClientCore.check`)	GAME-05

Task	Backlog ID
`ValidationResult.failureReason` propagation to UI via `GameStateUpdate` — Connect Four	GAME-05

Quality, Testing & Integration

Task	Backlog ID
Final README, architecture doc final revision, stub documentation	QA-05
Unit tests: `GameSessionManager`, `StatsManager`	QA-06
Unit tests: win detection (Tic-Tac-Toe, Connect Four)	QA-06
Integration test: full game flow (lobby → game start → turns → game over)	QA-06
Integration test: turn notification (`GameStateUpdate` routed to correct player's screen)	QA-06
Integration test: game-over stats update (`StatsManager` → `Database` stub)	QA-06
Merge all Sprint 3 branches into `develop`, then `main`	QA-07
Demo script verification (scripted walkthrough of all platform flows)	QA-07
End-to-end demo run (login → lobby → game → result → leaderboard)	QA-07

Dependency Summary

Task	Depends On
`CLIENT-02` (`ClientCore` full)	`STUBS-01` (NETWORK), `GAME-01` (clientGame interface), `CLIENT-01` skeleton
`SERVER-02` (`ServerCore` full)	`STUBS-01` (NETWORK, Database), `SERVER-01` skeleton
`AUTH-01` (`SessionManager`)	`STUBS-01` (CredentialsDatabase), `MODEL-01` (Player)
`LOBBY-01` (`GameLobby`)	`SERVER-02`, `MATCH-01`
`MATCH-01` (`matchmaking`)	`MODEL-01` (Player), `STUBS-01` (Database)
`CHAT-01` (`Chat`)	`LOBBY-01`
`SESSION-01` (`Game` service + `GameEngine`)	`GAME-01` (Game interface), `EVENTS-01` (GameStateUpdate)
`GAME-02`, `GAME-03`	`GAME-01` (Game, RuleValidator, PlayerAction)

Task	Depends On
`UI-02` , `UI-03` , `UI-04`	`UI-01` , `MODEL-01` (Player), `EVENTS-01` (GameStateUpdate)
`LEADER-01` (skeleton)	`MODEL-01` (GeneralStats, Rank), `STUBS-01` (Database)
`SESSION-02` (full pipeline)	`SESSION-01` , `SERVER-02` , `EVENTS-01`
`STATS-01`	`MODEL-01` , `STUBS-01` , `LEADER-01` , `SESSION-02`
`GAME-04` , `GAME-05`	`GAME-02` , `GAME-03` , `GAME-01` (clientGame interface)
`UI-05`	`SESSION-02` , `CLIENT-02` (ClientReceiver), `UI-04`
`UI-06`	`STATS-01` , `UI-05` , `UI-02` , `UI-03`
`QA-06` , `QA-07`	All Sprint 3 feature branches

Product Backlog

Each ID = one feature branch (`ID/short-description`) = one merge request into `develop` . No ID spans more than one sprint.

Priority: `High` — required for a working demo. `Medium` — improves completeness. `Low` — deferred if time is short.

Sprint 1 — Foundation

ID	Branch	Feature Scope	Priority	Sub-Team
INFRA-01	`INFRA-01/project-setup`	Package structure, GitLab repo, README, branching conventions	High	QT&I / Platform Core
STUBS-01	`STUBS-01/network-db-stubs`	`NETWORK` stub (TCP), `Database` stub, `CredentialsDatabase` stub	High	Platform Core
EVENTS-01	`EVENTS-01/game-state-update`	`GameStateUpdate` , `GameStatus` enum	High	Platform Core
CLIENT-01	`CLIENT-01/client-core-skeleton`	`ClientCore` skeleton + `ClientReceiver` skeleton (stubs only)	High	Platform Core
SERVER-01	`SERVER-01/server-core-skeleton`	`ServerCore` skeleton (stubs only)	High	Platform Core

ID	Branch	Feature Scope	Priority	Sub-Team
GAME-01	`GAME-01/game-interfaces`	`Game`, `RuleValidator`, `PlayerAction`, `clientGame` interfaces & abstractions	High	Rules & Validation
MODEL-01	`MODEL-01/player-domain`	`Player`, `GeneralStats`, `PlayerStats`, `Rank`, `Settings`	High	Rules & Validation
UI-01	`UI-01/auth-screens`	`WelcomePage`, `AuthenticationScreen`, `LoginPage`, `State` enum	High	Client/UI
QA-01	`QA-01/sprint1-tests`	JUnit infrastructure + unit tests for all Sprint 1 classes	High	QT&I

Sprint 2 — Core Features

ID	Branch	Feature Scope	Priority	Sub-Team
QA-02	`QA-02/sprint1-merge`	Merge Sprint 1 into `develop`, smoke test	High	QT&I
CLIENT-02	`CLIENT-02/client-core-full`	`ClientCore + ClientReceiver` full implementations	High	Platform Core
SERVER-02	`SERVER-02/server-core-full`	`ServerCore` full routing implementation	High	Platform Core
AUTH-01	`AUTH-01/session-manager`	`SessionManager` — full token lifecycle (create, store, invalidate)	High	Platform Core
LOBBY-01	`LOBBY-01/game-lobby`	`GameLobby` service — full implementation	High	Platform Core
MATCH-01	`MATCH-01/matchmaking`	`matchmaking` service — full implementation	High	Platform Core
CHAT-01	`CHAT-01/chat-service`	`Chat` service — full implementation	High	Platform Core
SESSION-01	`SESSION-01/game-engine`	`Game` service skeleton + `GameEngine` interface	High	Platform Core
LEADER-01	`LEADER-01/leaderboard-skeleton`	`Leaderboard` service skeleton	Medium	Platform Core
GAME-02	`GAME-02/tictactoe`	Tic-Tac-Toe — `Game`, `PlayerAction`, `RuleValidator`, `GameState` (win detection in Sprint 3)	High	Rules & Validation

ID	Branch	Feature Scope	Priority	Sub-Team
GAME-03	`GAME-03/connect-four`	Connect Four — `Game`, `PlayerAction`, `RuleValidator`, `GameState` (win detection in Sprint 3)	High	Rules & Validation
UI-02	`UI-02/home-dashboard`	`HomePage`, `MainGameDashboard`	High	Client/UI
UI-03	`UI-03/profile-dashboard`	`ProfileDashboard`	Medium	Client/UI
UI-04	`UI-04/game-screens`	`PlayGame`, `GameGUI`, `ChatboxGUI`	High	Client/UI
QA-03	`QA-03/arch-update`	Architecture doc update for Sprint 2	High	QT&I
QA-04	`QA-04/sprint2-tests`	Unit + integration tests for all Sprint 2 classes and flows	High	QT&I

Sprint 3 — Integration & Demo

ID	Branch	Feature Scope	Priority	Sub-Team
SESSION-02	`SESSION-02/move-pipeline`	`GameSessionManager` full pipeline + `Game` service full impl + full event wiring	High	Platform Core
STATS-01	`STATS-01/stats-leaderboard`	`StatsManager` + `Leaderboard` full implementation (MMR, `getTopPlayersByRank`)	Medium	Platform Core
GAME-04	`GAME-04/ttt-win-detection`	TTT win detection + `clientGame` impl + `ValidationResult` failure propagation	High	Rules & Validation
GAME-05	`GAME-05/c4-win-detection`	Connect Four win detection + `clientGame` impl + failure propagation	High	Rules & Validation
UI-05	`UI-05/turn-alerts`	Turn alert display + win/loss/tie result screen	High	Client/UI
UI-06	`UI-06/ui-polish`	`Settings` UI + player stats view + leaderboard view + full navigation wiring	Medium	Client/UI
QA-05	`QA-05/final-docs`	Final README + architecture doc revision + stub documentation	High	QT&I
QA-06	`QA-06/sprint3-tests`	Integration tests (full game flow, notifications, stats) + remaining unit tests	High	QT&I

ID	Branch	Feature Scope	Priority	Sub-Team
QA-07	`QA-07/demo-release`	Final merge into `develop` → `main`, demo script verification, end-to-end demo run	High	QT&I

Repo Structure:

```
seng300-202601-pg-5/
├── .mvn/
│   └── wrapper/
├── docs/
├── src/
│   ├── main/
│   │   └── java/ca/ucalgary/seng300/
│   │       ├── platform/
│   │       │   ├── stubs/
│   │       │   ├── client/
│   │       │   ├── server/
│   │       │   └── services/
│   │       ├── model/
│   │       ├── game/
│   │       │   ├── core/
│   │       │   ├── tictactoe/
│   │       │   └── connect4/
│   │       └── ui/
│   │           ├── auth/
│   │           ├── main/
│   │           └── game/
│   └── test/
│       └── java/ca/ucalgary/seng300/
│           ├── platform/
│           ├── model/
│           ├── game/
│           └── ui/
├── .gitignore
├── README.md
├── pom.xml
├── mvnw
└── mvnw.cmd
```

Plan version: aligned with architecture document v1.1.0 (updated Platform Core design). Adjust task assignments to individual team members at the start of each sprint. No backlog ID spans more than one sprint.